Lecture 13 · Part III · Lecture notes

Transfer learning

Dataset: Flowers102

How to use these notes

The primary text for this lecture is Géron, Chapter 12. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 The condition, named **2**

1.1 What a pretrained backbone has actually learned 2

2 Freeze, unfreeze, and augment **3**

2.1 Probe first, fine-tune second 3

2.2 The backbone, and its preprocessing 4

2.3 Letting the last block move 5

2.4 Augmentation 6

3 A failure condition: one transform, two loaders **6**

4 The ablation **7**

4.1 When the recipe does not work 7

5 The notebook **8**

6 Design exercise: an agritech company's plant-disease app **8**

7 Exercises **10**

1 The condition, named

Model	Test accuracy
Always the commonest species	3.87%
Convolutional network from scratch, 4,807,494 parameters, 20 epochs	9.82%
What the operator needs	90%

Lecture 12 built that network, and this lecture's notebook rebuilds it and retrains it for 20 epochs; it is not close.

The diagnosis, in one sentence

We asked 1,020 labelled images to pay for 4,807,494 parameters, most of which encode facts about *photographs in general* — edges, textures, colour gradients — rather than facts about flowers.
The repair is therefore not a better architecture, a longer run or a cleverer schedule. It is starting from weights that already work.

The measurement that names the problem is the gap. Retrained here for 20 epochs, the network scores 27.75% on the training set against 11.67% on the validation set, 16 points apart. That shape has a name — a gap between the images the network has seen and the ones it has not is *variance*. It is not a capacity problem: twenty epochs is a short run and the training accuracy is still climbing when it stops, but it is already 16 points ahead of validation, and fitting harder widens that gap rather than closing it.

Variance has two cures. More training data: no, labelling is the whole cost. More constraint on the hypothesis space: yes, and we have done it once already — weight sharing was a constraint and it bought a factor of 5,478,275. The question is where the next constraint comes from.

1.1 What a pretrained backbone has actually learned

Not "flowers". A hierarchy, and only the bottom of it is general.

Depth	What the filters respond to	Transfers?
first block	edges, colour opponency, simple gradients	to essentially anything
middle blocks	textures, repeated motifs, part-like shapes	to most natural images
last blocks	object-specific configurations	only to similar objects
the head	the 1,000 ImageNet classes	not at all — it is replaced

Which is the whole recipe: keep the bottom, adapt the top, replace the head.

Freezing is not a compromise forced by compute

It is a statement that those layers already encode what you would otherwise be trying to learn from 1,020 images.

The first layer of a ResNet-18 trained on ImageNet shows oriented light–dark boundaries at every angle and opponent-colour blobs — 9,408 numbers, and not one of them is about flowers, or about any of the 1,000 classes that network was fitted on. They are about *photographs*.

ImageNet is 1.28 million labelled photographs over 1,000 classes, none of them our species. Somebody already spent that on learning edges, colours, textures and parts, and those weights are a download. Our 1,020 labels should be spent on the only thing specific to us: which combination of parts is which species.

2 Freeze, unfreeze, and augment

Not examinable — engineering. Which backbone? MobileNet or EfficientNet-B0 (about 5 M parameters) when the budget is CPU or inference has to be fast; ResNet-18 or ResNet-50 (11 or 25 M) as the well-understood default; ConvNeXt or ViT-B (about 88 M) with plenty of data and compute. The names change every year; the trade-off does not. A bigger backbone is not automatically better on a small corpus — more features means more parameters in the head and more chance to overfit 1,020 images — so start small and grow it only if the validation curve says the features are the bottleneck.

2.1 Probe first, fine-tune second

Two decisions, in this order. A **linear probe** freezes everything, extracts features once and fits a linear classifier — cheap, and it tells you whether the backbone understands your images at all. Then **fine-tune**: unfreeze the last block or two at a small learning rate, and only if the probe was promising.

Why the order is not arbitrary

The probe is a measurement of the *features*, uncontaminated by anything you do to them. If it barely beats the trivial baseline, the features are wrong for this domain and fine-tuning is polishing the wrong object. If it is already close, fine-tuning has little left to buy and may cost you generalisation.

The recipe

1. Load a pretrained backbone and its preprocessing — the normalisation constants are part of the weights.
2. Freeze it, put it in `eval()` mode, extract features once.
3. Fit a linear head. Measure. This is the number everything else has to beat.
4. Unfreeze the last block, with a learning rate an order of magnitude smaller than the head's. Measure again.
5. Add augmentation, one transform at a time, each justified by an invariance you can state. Measure each.
6. Report the ablation, not just the best number.

Steps 3, 4 and 5 each end in *measure*. A recipe without them is a ritual.

2.2 The backbone, and its preprocessing

Transfer learning, as a sentence: take a network trained on a large labelled corpus, keep everything except its classifier, and train a new classifier on your own small corpus. The kept part is the **backbone** or **feature extractor**; what it produces for an image is that image's **representation** — a vector, learned by somebody else.

The backbone here is ResNet-18 (residual networks are Chapter 12's own example architecture): 11,689,512 parameters, trained on ImageNet, producing 512 numbers before its classifier.

Failure condition — the weights come with their own preprocessing

`ResNet18_Weights.DEFAULT.transforms()` specifies a resize to 256, a centre crop to 224, and per-channel normalisation with mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$.
Use *these* statistics, not ones computed from your own training images. The network's first layer expects inputs on the scale it was trained with, and substituting your own is a silent, uncrashing degradation.

Freezing is two lines — set `requires_grad = False` on every parameter, then replace `net.fc`, because a newly constructed module has `requires_grad=True` and so the head is trainable and nothing else is. That leaves 52,326 trainable parameters.

And `requires_grad = False` is *not* the same as `eval()`.

Failure condition — freezing weights does not freeze a network

Batch-norm running statistics are *buffers*, updated in the forward pass whenever the module is in training mode. No gradient is involved, so `requires_grad = False` does nothing to them. A "frozen" backbone in `train()` mode quietly replaces ImageNet's statistics with the statistics of 1,020 flower photographs, in batches of 32. The weights are fixed and the network still drifts — towards a corpus of 1,020 images, which is the opposite of what you froze it for. The fix is `eval()` on the frozen part: the same call that switches dropout off for evaluation, for the same underlying reason.

If nothing before the head is training, the backbone's output for a given image never changes — so run it once and fit the head on cached features. That is not an optimisation; it is a consequence of freezing. The sixty epochs of the head then run on a table of features, which costs almost nothing.

Model	Test accuracy
From scratch, 20 epochs	9.82%
Frozen backbone, linear head	81.09%

71.3 points better, and cheaper: each image passes through the backbone once. Why a linear model on somebody else's features beats your network: the 52,326 trainable parameters are fitted with 1,020 examples, about 51 parameters per image against 4713 before, while the 11,176,512 frozen ones were fitted with 1.28 million examples by somebody else.

The variance problem was not solved

It was *moved* — to a dataset large enough to absorb it.

2.3 Letting the last block move

The early layers hold edges and colours, which are not about flowers; the late layers hold parts and textures, which partly are. Unfreeze `layer4` and give it a learning rate an order of magnitude below the head's. One optimiser, two parameter groups — that is all a differential learning rate is.

Part of resnet18	Parameters
`conv1` through `layer3`, frozen	2,782,784
`layer4` and the new head, trained	8,446,054

The two rates have to differ because the head starts random and has everything to learn, while the pretrained block is already good and has something to lose. Give the backbone the head's rate and the first few batches — whose gradients come through a randomly initialised head — will damage the representation you downloaded. It does not crash; the accuracy simply comes out lower and you conclude that transfer learning does not work.

Failure condition — catastrophic forgetting

The pretrained weights being a good starting point is exactly what makes them fragile: a step large enough to be useful on random weights is far too large for weights that are already nearly right.

Fine-tune the whole backbone at the rate you would use to train from scratch and the first gradient step overwrites the features you came for; the network then relearns them from 1,020 images, badly. This is why differential learning rates exist, and why the usual recipe warms up the new head first while everything else is frozen: the head starts random, and its early gradients are large enough to damage anything they reach.

2.4 Augmentation

We cannot buy labels; we can show the network a different view of the same labelled image every epoch. A random resized crop teaches scale and position tolerance; a horizontal flip says a flower is still that species mirrored. Each is a transformation *the label is known to survive*, and that is the entire criterion — a modelling decision, not a library call.

Augmentation belongs in the training pipeline only: random resized crop (scale 0.55 to 1) and horizontal flip for training; for validation *and* test, no randomness at all. (The notebook decodes every image straight to 224×224; the weights' own recipe resizes to 256 and centre-crops to 224.) Two pipelines, deliberately — §3 is what happens with one. With layer4 unfrozen and augmentation on, the model reaches 87.82% test accuracy, against 9.82% from scratch: not cheaper — a larger network on larger images, with two validation passes an epoch included — but far better.

3 A failure condition: one transform, two loaders

It is easy to build one pipeline and hand it to both loaders. The training loader wants random crops and flips; the evaluation loader must not have them.

Failure condition — an augmented evaluation set is not a measurement

It makes the score a *random variable* rather than a function of the weights. Score one fixed model — the fine-tuned one — on the clean validation set and it gives 90.88%; on the augmented set, ten times, the mean is 89.01% with a spread of 2.16 points.
The same weights, the same set, 2.16 points apart — and a deterministic function of fixed weights and fixed data does not wobble. Those 2.16 points of noise are a third of the gain that fine-tuning added on top of the frozen probe. Nothing raises; the only symptom is that the number moves when nothing else did.

In this eight-epoch run both curves are still rising and both peak at the last epoch, so early stopping keeps the same checkpoint either way. Where a curve flattens, 2.16 points of noise is enough to order two epochs by the evaluator's seed, and early stopping on the augmented one then keeps the wrong checkpoint.

The check, in two lines

```
a = evaluate(model, val_loader)
b = evaluate(model, val_loader)
assert a == b, f"evaluation is not deterministic: {a} vs {b}"
```

A comment saying "do not augment the validation set" is read once. This runs every time, costs two forward passes, and fails loudly at the moment the mistake is made rather than three lectures later — and it catches all three causes: augmentation, dropout still active, and a shuffled loader whose batches are averaged rather than pooled.

4 The ablation

Change	Test accuracy	Gain
from scratch	9.82%	—
+ pretrained backbone, frozen	81.09%	71.3 pts
+ `layer4` unfrozen, and augmentation	87.82%	6.7 pts

Almost all of the gain is in one row, and saying *which* row is what makes the result useful to somebody else.

All three rows are single runs, and none is replicated. The notebook unfreezes `layer4` and augments in the same run, so it cannot say how those 6.7 points divide between the two; that takes one more fine-tune, without augmentation.

Model	Test accuracy
Uniform guess	0.98%
Commonest species	3.87%
Convolutional net, from scratch	9.82%
Frozen backbone, linear head	81.09%
Fine-tuned + augmented	87.82%
Operator's requirement	90%

Be precise about "cheaper". The from-scratch network trains all 4,807,494 of its parameters for 20 epochs on 128×128 images. The frozen probe sends each image through the backbone once, without gradients, and trains 52,326 parameters on a table. The fine-tune is no cheaper than from scratch: fewer epochs, eight, but a much larger network on 224×224 images, gradients through `layer4`, and the validation set scored twice an epoch. The frozen probe is the cheap one — and it is also the one you should run first, every time.

4.1 When the recipe does not work

Your images are unlike the corpus the backbone was trained on — radar, spectrograms, microscopy — and the learned features are about natural photographs. Your task is not classification, and the backbone was trained to discard position. Or you have millions of labelled examples, and then the constraint is costing you rather than helping: transfer is strongest exactly where labels are scarce.

How to tell, cheaply: fit a linear head on frozen features and compare with the trivial baseline. If the frozen features barely beat it, the backbone does not understand your images and no amount of fine-tuning will hide it.

5 The notebook

What to change

Freeze everything including the final block and refit. The accuracy drops by a few points — which tells you how much of the gain came from adapting the last block rather than from the backbone itself, and separates the two rows of the ablation that are easiest to conflate.

6 Design exercise: an agritech company's plant-disease app

The lecture closes with a design exercise in the manner of an ML system-design interview, on the ML system behind a company's product, with the shape of data this method suits: few labels, many pixels.

The task. An agritech company sells a crop-protection app: a farmer photographs a leaf with a phone, and the app names the disease — or says healthy — and recommends a treatment, so that the crop is treated before the disease reaches the neighbours' fields. The feature at the centre is the classifier behind that screen: one class from a fixed list of diseases, on a phone in a field. The users are the farmers and the company's field agronomists; the business measures renewals, the cost of a wrong recommendation — a spray that was not needed, a blight that was missed — and its exposure where advice on crop-protection products is regulated. Given: a few dozen labelled photographs per disease, most taken in the company's trial plots by its agronomists; a trickle of field photographs from the pilot customers; a backbone pretrained on a large photographic corpus. Tens of thousands of farmers, many phone models, every kind of light and background. The output feeds the farmer's screen, the agronomists who follow up, and the outbreak map the company sells to cooperatives.

1. **Frame it.** One row per photograph; the label is the disease, or healthy; the output is one class from the list. The metric is per disease — the recall of each, since a missed blight spreads and a false alarm costs a spray and the farmer's trust — against the baseline "healthy", the commonest label. Count the labels per class before choosing anything: a few dozen per disease is not a from-scratch budget. *Pitfall:* few labels, planned as if there were many. The product team's first plan budgets a network trained from scratch, a labelling campaign to feed it, and a launch date after it; from scratch on a few dozen images per class the network memorises the training set and scores near the majority baseline on everything else, and the campaign spends the agronomists' hours where they buy least. The method for this budget is a pretrained backbone, and the agronomists' hours go to the validation set, where a label is worth most.

2. **The data, and where it comes from.** The trial plots' photographs: one camera, a white sheet behind the leaf, even light. The customers': many phones, soil and sky behind the leaf, hard shadow, dust, a thumb in the corner. The backbone's early layers — edges, colour, texture — transfer to both; the task is to make the labelled set look like the app's input. *Pitfall:* domain shift from phone cameras and fields. Fit and validate on the trial plots' photographs and the probe looks excellent; in the field the model meets a distribution it never saw, the app is wrong in the one place the customer uses it, and the first support tickets say so. Label field photographs from the pilot customers, even fewer of them, and let the trial plots' images be

training data only: the number in the release note is the number on farmers' photographs.

3. **Split by farmer, not by photograph.** A farmer photographs the same plant several times, from the same phone, in the same light, so the photographs are near twins. Hold whole farmers out: the validation set is farmers — their phones, fields and light — the model never saw, and the test set is another such group, read once at the end. *Pitfall:* a random split of photographs. The same diseased plant lands on both sides, the score measures whether the model recognises a plant it has already seen, and the first pilot customers say otherwise. Split by farmer, and check that no phone model, field or day of the held-out group appears in training.

4. **Representation: the backbone's features, with its own preprocessing.** The representation of a leaf is the vector the pretrained backbone emits before its classifier: a few hundred numbers, learned by somebody else on a corpus a thousand times larger. The weights ship with their preprocessing — the resize, the centre crop, the per-channel mean and standard deviation — and those constants are part of the weights, in the phone build as much as in the training notebook. *Pitfall:* your own normalisation. Standardising with the mean and spread of your own leaf images feeds the first layer inputs on a scale it was not trained for; nothing raises, and the probe comes out worse. The same when the app engineers feed the phone's full-resolution image unresized. Use `weights.transforms()` and nothing else, in the app exactly as in training.

5. **The model: probe frozen first, then the last block at its own rate.** Freeze the backbone, put it in `eval()` mode, extract the features once and fit a linear head on the table: cheap, and the number everything after has to beat. If the probe is far above the baseline, unfreeze the last block with a learning rate an order of magnitude below the head's, and keep the frozen batch-norm layers in `eval()`, because freezing gradients does not freeze their running statistics. *Pitfall:* fine-tuning first, and everything at once. The whole backbone at the head's learning rate: the first batches' gradients come through a random head and overwrite the features you downloaded, the network relearns them from a few hundred leaves, badly, and the team concludes that transfer does not work and asks for the labelling campaign after all. Probe, measure, then move only the last block, slowly.

6. **Failure condition: augmentation, and the loader it belongs to.** Manufacture views of the training leaves: rotations and flips, since a leaf has no up; random crops; brightness, since phones differ. Not hue: a yellowing *is* the disease, and the label does not survive a colour shift. Two pipelines, deliberately: the evaluation loader has no randomness in it. *Pitfall:* one transform handed to both loaders. The validation score becomes a random variable — the same weights scored twice give two numbers, pessimistic on average and noisy by points — and a release is signed off on a number that moves. Run every evaluation twice and assert that the two agree; the assert fails at the moment the mistake is made, and catches dropout left on and a shuffled loader as well.

7. **Evaluation: stop on the right curve, read the test farmers once.** Choose the epoch to keep on the clean validation curve of held-out farmers. Report the ablation — probe, then the last block, then augmentation — because the row the gain came from is what the product team needs when the next crop is added. Then the test farmers, once, with the sample size beside the number in the release note. *Pitfall:* early stopping on the wrong curve. The training curve rises for as long as you let it and selects the last epoch; an augmented validation curve

carries enough noise to order two epochs by the evaluator's seed, so where it flattens the checkpoint kept is the wrong one. Keep the checkpoint by the clean validation curve, and by nothing else.

8. **In the field.** Photographs arrive with the season: new crops, new diseases, new phones. The frozen probe is cheap enough to rerun on each month's labelled field photographs, to see whether the features still hold. Release to a share of farmers first; the diagnoses the agronomists confirm on their visits are the truth, so retrain on them, monitor per-disease recall and the share of photographs the app could not name, and keep an agronomist on call for the outbreak alerts. *Pitfall:* the test set reread at every release. Each version is compared on the same test farmers; after a dozen readings the test set has chosen the model, and it is a validation set with a better name. Retire it: each release is scored once on farmers who arrived since the last one, and the old test group joins training.

9. **Ship it.** Labels counted → field photographs labelled → split by farmer → the weights' own preprocessing → frozen probe, measured → last block unfrozen slowly, batch norms in `eval()` → augmentation on the training loader only → early stopping on the clean curve, ablation reported → test farmers once → a share of farmers first, retrain on confirmed diagnoses.

7 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 14's deck and in the exercise book.

1. State the order in which a pretrained network's head must be replaced and its body frozen, and what goes wrong in the other order. [4]

2. Fine-tuning uses 10^{-4} on the pretrained block and 10^{-3} on the new head. Justify the difference in one sentence. [4]

3. Freezing a batch-norm layer's weights does not freeze the layer. State what still changes, and the call that stops it. [4]

4. An augmented validation loader is used to select the epoch to keep. State the two things this costs, and the one-line test that detects it. [5]

5. A frozen-backbone probe is both more accurate and faster than training from scratch on 1,020 images. Explain why both, and name the cost that must still be counted. [5]

Summary

The from-scratch network reached 9.82% because 1,020 labelled images were being asked to pay for 4,807,494 parameters, most of which encode facts about photographs rather than about flowers. The gap of 16 points between training and validation is variance, and with labelling as the dominant cost the only available cure is more constraint.

A frozen backbone and a linear head reached 81.09% — 71.3 points better, and cheaper, since each image passes through the backbone once — because the trainable parameters fell to about 51

per labelled image while 11,176,512 frozen ones had been fitted on 1.28 million photographs by somebody else. The variance problem was not solved but moved. Unfreezing the last block with augmentation added 6.7 points more, in a single run whose split between the two is not measured.

Three failure conditions on the way, none of which raises: preprocessing with your own normalisation instead of the weights' own; a frozen backbone left in `train()` mode, whose batch-norm buffers drift towards 1,020 flowers because they are not parameters and freezing gradients does not touch them; and one transform handed to both loaders, which cost 2.16 points of noise and could have selected a different epoch, though in this run it did not. The closing design exercise built the plant-disease app an agritech company sells to farmers in nine steps — a label budget that chose transfer, field photographs for validation in place of the trial plots', a split by farmer, the weights' own preprocessing, the probe before the fine-tune, augmentation on the training loader only, early stopping on the clean curve, and a test group retired after one reading — each step named for the pitfall it avoids.